COMP1117 Tutorial 9

Coding Practice (Functions)

From concepts to code

You already know

Variables and expressions
if statements and loops
Strings and lists

Today

Turn repeated logic into functions
Return useful results
Test each function with examples

Quick check: open a new file named tutorial9.py and run one print statement.

Work in pairs when useful. Each person should still type and run the code.

Today's plan

01 **Recall**
Read a function header and trace one call.

02 **Build**
Write short functions with parameters and return values.

03 **Combine**
Call one helper function from another function.

04 **Test**
Use normal, boundary, and unusual inputs.

Main rule: a function should do one clear job.

Anatomy of a function

EXAMPLE

```
def rectangle_area(width, height):
    area = width * height
    return area

result = rectangle_area(4, 3)
print(result)
```

Output: 12

Function header

def, a name, parameters, and a colon

Function body

Indented statements run when the function is called.

Function call

The arguments 4 and 3 become width and height.

Parameters, arguments, and return values

Parameters

Names in the function definition.

width and height

Arguments

Values supplied in a call.

4 and 3

Return value

A value sent back to the caller.

12

```
value = rectangle_area(4, 3)
```

print displays a value. return gives a value back to the caller.

A function design checklist

Before coding

1. What single job should it do?
2. What inputs does it need?
3. What value should it return?

Before moving on

1. Did I call the function?
2. Did I test a boundary case?
3. Does every path return a value?

Choose names that explain the job: count_vowels is clearer than cv.

Quick check: trace the calls

TRACE WITHOUT RUNNING

```
def add_tax(price, rate):
    return price * (1 + rate)

first = add_tax(100, 0.1)
second = add_tax(first, 0.05)
print(second)
```

Questions

1. What is first?

2. What arguments are used in the second call?

3. What is printed?

Discuss for 90 seconds before running the code.

Quick check: answer

FIRST CALL

```
first = add_tax(100, 0.1)
      = 100 * 1.1
      = 110.0
```

SECOND CALL

```
second = add_tax(first, 0.05)
       = add_tax(110.0, 0.05)
       = 115.5
```

Printed output: 115.5

The return value from the first call becomes an argument in the second call.

A function can be reused with different values without rewriting its logic.

Task 1: Rectangle metrics

Return both the area and perimeter of a rectangle.

```
rectangle_metrics(w, h)
```

Requirements

- area = width × height
- perimeter = 2 × (width + height)
- return the two values as a tuple

EXPECTED USE

```
area, perimeter = rectangle_metrics(4, 3)
print(area)       # 12
print(perimeter)  # 14
```

Time: 4 minutes

Task 1: Answer

ONE POSSIBLE SOLUTION

```
def rectangle_metrics(width, height):
    area = width * height
    perimeter = 2 * (width + height)
    return area, perimeter

area, perimeter = rectangle_metrics(4, 3)
print(area, perimeter)
```

Check

rectangle_metrics(1, 1)
returns (1, 4)

rectangle_metrics(5, 2)
returns (10, 14)

The caller can unpack the returned tuple into two variables.

Task 2: Temperature converter

Convert a Celsius temperature to Fahrenheit.

```
celsius_to_fahrenheit
(c)
```

FORMULA

```
F = C * 9 / 5 + 32
```

Tests to pass

celsius_to_fahrenheit(0) returns 32.0
celsius_to_fahrenheit(100) returns 212.0
celsius_to_fahrenheit(-40) returns -40.0

Return the result. Do not print inside the function.

Task 2: Answer

ONE POSSIBLE SOLUTION

```
def celsius_to_fahrenheit(celsius):
    fahrenheit = celsius * 9 / 5 + 32
    return fahrenheit

print(celsius_to_fahrenheit(0))
print(celsius_to_fahrenheit(100))
print(celsius_to_fahrenheit(-40))
```

OUTPUT

```
32.0
212.0
-40.0
```

-40 is a useful test because both temperature scales have the same value there.

Task 3: Delivery fee

Calculate a delivery fee from weight and service type.

```
delivery_fee(weight, express)
```

Rules

Base fee: $25
Add $8 for each kg above 2 kg
Add $15 when express is True

EXPECTED RESULTS

```
delivery_fee(1.5, False)  # 25
delivery_fee(4, False)    # 41
delivery_fee(4, True)     # 56
```

Time: 5 minutes

Task 3: Answer

ONE POSSIBLE SOLUTION

```
def delivery_fee(weight, express):
    fee = 25

    if weight > 2:
        fee = fee + (weight - 2) * 8

    if express:
        fee = fee + 15

    return fee
```

Boundary tests

delivery_fee(2, False) → 25

delivery_fee(2.5, False) → 29

delivery_fee(2, True) → 40

Two independent if statements allow both surcharges to apply.

Task 4: Count vowels

Count vowels in a string, ignoring letter case.

```
count_vowels(text)
```

Rules

Vowels: a, e, i, o, u
Uppercase and lowercase count
Return an integer

EXPECTED RESULTS

```
count_vowels("Hello")       # 2
count_vowels("FUNCTIONS")   # 3
count_vowels("rhythm")      # 0
```

Hint: loop through text.lower().

Task 4: Answer

ONE POSSIBLE SOLUTION

```
def count_vowels(text):
    count = 0

    for character in text.lower():
        if character in "aeiou":
            count = count + 1

    return count
```

Trace "Hello"

```
h  count 0
e  count 1
l   count 1
l   count 1
o  count 2
```

The accumulator count changes only when the condition is true.

Task 5: List average

Return the average of a list of numbers.

```
list_average(numbers)
```

Requirements

Use a loop to calculate the total.
Return None for an empty list.
Otherwise return total / length.

EXPECTED RESULTS

```
list_average([2, 4, 6])  # 4.0
list_average([5])        # 5.0
list_average([])         # None
```

Handle the empty list before dividing.

Task 5: Answer

ONE POSSIBLE SOLUTION

```
def list_average(numbers):
    if len(numbers) == 0:
        return None

    total = 0
    for number in numbers:
        total = total + number

    return total / len(numbers)
```

Why early return?

The empty-list case finishes immediately.

The remaining code can assume the list contains at least one value.

None means that no average exists for the empty input.

Task 6: Prime checker

Return True when n is prime and False otherwise.

```
is_prime(n)
```

Prime number

An integer greater than 1
with no divisor except 1 and itself.

0, 1, and negative values are not prime.

EXPECTED RESULTS

```
is_prime(2)   # True
is_prime(9)   # False
is_prime(17)  # True
is_prime(1)   # False
```

Hint: if n % divisor == 0, a divisor has been found.

Task 6: Answer

ONE POSSIBLE SOLUTION

```
def is_prime(n):
    if n < 2:
        return False

    divisor = 2
    while divisor * divisor <= n:
        if n % divisor == 0:
            return False
        divisor = divisor + 1

    return True
```

Return paths

n < 2: return False

Divisor found: return False

No divisor found: return True

Testing divisors only up to √n is enough.

Task 7: Grade report

Build a report by calling one function from another.

```
grade_report(name, mark)
```

Letter grade rules

A: mark ≥ 80
B: mark ≥ 70
C: mark ≥ 60
D: mark ≥ 50
F: otherwise

Write two functions:

letter_grade(mark)
grade_report(name, mark)

EXPECTED RESULT

```
grade_report("Ada", 84)
# returns "Ada: 84 (A)"
```

Time: 7 minutes

Task 7: Solution plan

01 **Helper**
letter_grade(mark) returns one letter.

02 **Main function**
grade_report(name, mark) calls the helper.

03 **Formatting**
Build the string: name: mark (letter).

STARTER

```
def grade_report(name, mark):
    letter = ____________________
    return f"{name}: {mark} ({letter})"
```

Task 7: Answer

HELPER FUNCTION

```
def letter_grade(mark):
    if mark >= 80:
        return "A"
    if mark >= 70:
        return "B"
    if mark >= 60:
        return "C"
    if mark >= 50:
        return "D"
    return "F"
```

MAIN FUNCTION

```
def grade_report(name, mark):
    letter = letter_grade(mark)
    return f"{name}: {mark} ({letter})"

print(grade_report("Ada", 84))
```

Ada: 84 (A)

Before you leave

Testing checklist

- A normal input
- A boundary value
- An empty or smallest input
- A result you can verify by hand

Common mistakes

- Calling before defining
- Missing return
- Printing instead of returning
- Forgetting indentation
- Testing only one case

Questions?